LIBXSTREAM

LIBXSTREAM is an OpenCL-based accelerator backend library providing streams, events, and device memory management for GPU offloading. It implements the DBCSR ACC interface, making it a drop-in OpenCL backend for CP2K/DBCSR.

Targets OpenCL devices across vendors (Intel, AMD, NVIDIA). Depends on LIBXS for utility infrastructure.

Build

LIBXS must be a sibling directory (controlled by `LIBXSROOT`). An OpenCL SDK must be available.

Library — build both LIBXS and LIBXSTREAM as separate libraries:

```
git clone https://github.com/hfp/libxs.git
git clone https://github.com/hfp/libxstream.git

cd libxs && make -j $(nproc)
cd ../libxstream && make -j $(nproc)
```

This produces `lib/libxstream.a` and `lib/libxstream.so`.

Header-only (explicit) — include `libxstream_source.h` (no separate library needed for either LIBXSTREAM or LIBXS). Safe to include from multiple translation units:

```
#include <libxstream/libxstream_source.h>
```

Header-only (implicit) — compile with `-DLIBXSTREAM_SOURCE`. Any LIBXSTREAM or LIBXS public header then automatically includes the implementation. No special include order is required. `-DLIBXSTREAM_SOURCE` implies `-DLIBXS_SOURCE`; LIBXS can also be made header-only independently with `-DLIBXS_SOURCE`.

The library is compiled for SSE4.2 by default but dynamically dispatches to the best ISA available at runtime (up to AVX-512). Use `SSE=0` to compile natively for the build host.

Variable	Default	Description
GNU	1	GNU GCC-compatible compiler (0: Intel)
DBG	0	Debug build
SYM	0	Include debug symbols (-g)
SSE	1	x86 baseline: 0=native, 1=SSE4.2 (portable)

pkg-config support: `lib/libxstream.pc`.

Installation

Install into a chosen prefix (LIBXS must be built first):

```
make -j $(nproc) install PREFIX=$HOME/libxstream
```

This installs headers and the static and shared libraries under `PREFIX`. The header-only source tree is optional; add `HEADER_ONLY=1` to install it.

Out-of-tree builds are also supported:

```
mkdir /tmp/libxstream-build && cd /tmp/libxstream-build
make -j $(nproc) -f /path/to/libxstream/Makefile
```

API

The public C API is declared in `libxstream/libxstream.h`. All implementation details are sealed behind opaque types.

Initialization

```
int libxstream_init(void);
int libxstream_init_config(const libxstream_init_config_t* cfg);
void libxstream_init_config_default(libxstream_init_config_t* cfg);
int libxstream_finalize(void);
```

`libxstream_init()` initializes with environment/defaults. `libxstream_init_config()` allows explicit control over USM level, device selection, verbosity, and sub-buffers before the runtime starts:

```
libxstream_init_config_t cfg;
libxstream_init_config_default(&cfg);  /* -1 = use env/default */
cfg.usm = 1;       /* 0:off, 1:Intel USM, 2:SVM coarse, 3:SVM caps */
cfg.device = 0;    /* device index (-1: env/first) */
cfg.verbosity = 2;
cfg.subbuffer = 1;  /* offset kernel-arguments (only used if USM is off) */
libxstream_init_config(&cfg);
```

A field left at `-1` takes the environment variable or the default; anything else overrides it. `cfg.subbuffer` corresponds to `LIBXSTREAM_SUBBUFFER` and only takes effect where a pointer argument cannot carry its own offset, i.e. with USM disabled — setting it alongside `cfg.usm = 0` is the case it exists for.

Passing NULL to `libxstream_init_config` is equivalent to calling `libxstream_init`.

Devices

```
int libxstream_device_count(int* ndevices);
int libxstream_device_set_active(int device_id);
int libxstream_device_sync(void);
```

Streams

```
int libxstream_stream_create(libxstream_stream_t** stream_p,
                             const char* name, int priority);
int libxstream_stream_destroy(libxstream_stream_t* stream);
int libxstream_stream_sync(libxstream_stream_t* stream);
int libxstream_stream_wait_event(libxstream_stream_t* stream,
                                 libxstream_event_t* event);
```

Events

```
int libxstream_event_create(libxstream_event_t** event_p);
int libxstream_event_destroy(libxstream_event_t* event);
int libxstream_event_record(libxstream_event_t* event,
                            libxstream_stream_t* stream);
int libxstream_event_query(libxstream_event_t* event,
                           libxstream_bool_t* has_occurred);
int libxstream_event_sync(libxstream_event_t* event);
```

Memory

Device and host memory allocation, transfers (H2D, D2H, D2D), and initialization:

```
int libxstream_mem_allocate(void** dev_mem, size_t nbytes);
int libxstream_mem_deallocate(void* dev_mem);
int libxstream_mem_host_allocate(void** host_mem, size_t nbytes,
                                 libxstream_stream_t* stream);
int libxstream_mem_host_deallocate(void* host_mem,
                                   libxstream_stream_t* stream);
int libxstream_mem_copy_h2d(const void* host_mem, void* dev_mem,
                            size_t nbytes, libxstream_stream_t* stream);
int libxstream_mem_copy_d2h(const void* dev_mem, void* host_mem,
                            size_t nbytes, libxstream_stream_t* stream);
int libxstream_mem_copy_d2d(const void* src, void* dst,
```

```
                                size_t nbytes, libxstream_stream_t* stream);
int libxstream_mem_zero(void* dev_mem, size_t offset, size_t nbytes,
                        libxstream_stream_t* stream);
```

DBCSR Compatibility

The header `libxstream/libxstream_dbcsr.h` provides the `c_dbcsr_acc_*` symbols expected by DBCSR, allowing LIBXSTREAM to serve as a drop-in accelerator backend.

CP2K Offload Interface

The header `libxstream/libxstream_cp2k.h` implements CP2K's offload runtime interface — the `offload*` functions for general GPU operations (memory, streams, events, synchronization).

Samples

Each sample has its own Makefile under `samples/`:

```
cd samples/smm && make
cd samples/ozaki && make
```

SMM — Small Matrix Multiplication

Implements the ACC LIBSMM interface for batched small matrix multiply and transpose on OpenCL devices. Includes an auto-tuning framework and pre-tuned parameter sets for A100, BMG, GH200, H100, Mi250, P100, PVC, and V100. See samples/smm/README.md.

Ozaki — High-Precision GEMM

Ozaki scheme for high-precision GEMM emulation, fully offloaded to OpenCL. Two schemes (mantissa slicing and CRT) with automatic detection of Intel XMX matrix engines. See samples/ozaki/README.md. The CPU-side GEMM wrapper is part of LIBXS Ozaki.

Stencil — BF16-DPAS Finite Difference

Seismic wave propagation (RTM/FWI) via dimension-split stencils computed as batched small GEMMs on BF16 DPAS/XMX tensor units. Achieves FP32 accuracy through Dekker splitting. Supports isotropic and TTI operators, multiple factorization methods (sparse, dense cascade, hybrid), and runtime kernel specialization via a thread-safe registry. Includes a scalar proxy for non-DPAS hardware. See samples/stencil/README.md.

License

BSD 3-Clause

LIBXSTREAM Domains

CP2K Offload Interface

`libxstream/libxstream_cp2k.h` implements CP2K's offload runtime interface — the hardware-abstraction layer that CP2K uses for GPU-accelerated operations beyond DBCSR (grid integration, PW operations, etc.). The original interface provides `static inline` wrappers for CUDA, HIP, and OpenCL; LIBXSTREAM replaces the OpenCL path with a dedicated translation unit (`src/libxstream_cp2k.c`) that routes directly through LIBXSTREAM's internal API.

Relationship to the DBCSR Interface

CP2K's code has two accelerator interfaces:

Interface	Header	Purpose
DBCSR ACC	`libxstream_dbcsr.h`	Sparse matrix operations (DBCSR library)
Offload Runtime	`libxstream_cp2k.h`	General offload (memory, streams, events, synchronization)

The DBCSR adapter (`src/libxstream_dbcsr.c`) uses opaque `void*` handles and translates to LIBXSTREAM's typed API. The offload runtime adapter does the same but uses CP2K's `offloadStream_t`/`offloadEvent_t` typedefs (also `void*`). Both share the underlying LIBXSTREAM implementation.

API

The header is self-contained (C99, no LIBXSTREAM headers required) and provides opaque handle types, an error-checking macro, and functions covering five domains.

```
typedef void* offloadStream_t;
typedef void* offloadEvent_t;
typedef int   offloadError_t;

#define offloadSuccess EXIT_SUCCESS
```

```
const char* offloadGetErrorName(offloadError_t error);
offloadError_t offloadGetLastError(void);
```

`offloadGetErrorName` maps error codes to OpenCL error strings via `libxstream_opencl_strerror`. `offloadGetLastError` consumes and clears the last recorded error.

The `OFFLOAD_CHECK` macro aborts on failure after printing the error name and source location:

```
OFFLOAD_CHECK(offloadMalloc(&ptr, nbytes));
```

```
void offloadStreamCreate(offloadStream_t* stream);
void offloadStreamDestroy(offloadStream_t stream);
void offloadStreamSynchronize(offloadStream_t stream);
void offloadStreamWaitEvent(offloadStream_t stream, offloadEvent_t event);
```

`offloadStreamCreate` creates a stream with default priority (`LIBXSTREAM_STREAM_DEFAULT`).

```
void offloadEventCreate(offloadEvent_t* event);
void offloadEventDestroy(offloadEvent_t event);
void offloadEventRecord(offloadEvent_t event, offloadStream_t stream);
void offloadEventSynchronize(offloadEvent_t event);
bool offloadEventQuery(offloadEvent_t event);
```

```
void offloadMalloc(void** ptr, size_t size);
void offloadFree(void* ptr);
void offloadMallocHost(void** ptr, size_t size);
void offloadFreeHost(void* ptr);
```

```
void offloadMemcpyAsyncHtoD(void* ptr_dev, const void* ptr_hst, size_t size, offloadStream_t stream);
void offloadMemcpyAsyncDtoH(void* ptr_hst, const void* ptr_dev, size_t size, offloadStream_t stream);
void offloadMemcpyAsyncDtoD(void* dst, const void* src, size_t size, offloadStream_t stream);
void offloadMemcpyHtoD(void* ptr_dev, const void* ptr_hst, size_t size);
void offloadMemcpyDtoH(void* ptr_hst, const void* ptr_dev, size_t size);
void offloadMemsetAsync(void* ptr, int val, size_t size, offloadStream_t stream);
void offloadMemset(void* ptr, int val, size_t size);
```

The synchronous variants (`offloadMemcpyHtoD`, `offloadMemcpyDtoH`, `offloadMemset`) pass a NULL stream. `offloadMemsetAsync` supports arbitrary fill values via `libxstream_opencl_memset`.

```
void offloadDeviceSynchronize(void);
```

```
void offloadMemcpyToSymbol(const void* symbol, const void* src, size_t count);
void offloadEnsureMallocHeapSize(size_t required_size);
```

These are CUDA-specific operations (constant-memory writes and device-heap sizing) that have no direct OpenCL equivalent. They are currently stubs guarded by assertions. CP2K's OpenCL path disables the GPU grid subsystem (`__NO_OFFLOAD_GRID`) that would call them.

See Also

- LIBXSTREAM API (`libxstream/libxstream.h`) — the underlying OpenCL backend API
- DBCSR ACC Interface — the DBCSR adapter layer
- CP2K offload_runtime.h — upstream interface definition

DBCSR ACC Interface

`libxstream/libxstream_dbcsr.h` implements the DBCSR ACC interface — the accelerator backend contract defined by the DBCSR sparse-matrix library used in CP2K. By providing this interface on top of LIBXSTREAM's OpenCL runtime, the library acts as a **drop-in OpenCL backend** for DBCSR: no changes to DBCSR or CP2K source code are required.

Purpose

DBCSR expects every accelerator backend to expose a flat C API with the `c_dbcsr_acc_*` prefix covering five domains:

Domain	Functions
Initialization	`c_dbcsr_acc_init`, `c_dbcsr_acc_finalize`
Devices	`c_dbcsr_acc_get_ndevices`, `c_dbcsr_acc_set_active_device`, `c_dbcsr_acc_device_synchronize`
Streams	`c_dbcsr_acc_stream_create`, `c_dbcsr_acc_stream_destroy`, `c_dbcsr_acc_stream_sync`, `c_dbcsr_acc_stream_wait_eve`
Events	`c_dbcsr_acc_event_create`, `c_dbcsr_acc_event_destroy`, `c_dbcsr_acc_event_record`, `c_dbcsr_acc_event_query`, `c_d`
Memory	`c_dbcsr_acc_dev_mem_allocate`, `c_dbcsr_acc_dev_mem_deallocate`, `c_dbcsr_acc_dev_mem_set_ptr`, `c_dbcsr_acc_host`

Every function delegates to the corresponding `libxstream_*` routine (e.g., `c_dbcsr_acc_stream_create` calls `libxstream_stream_create`), translating between DBCSR's opaque `void*` handles and LIBXSTREAM's typed `libxstream_stream_t` / `libxstream_event_t` pointers.

Profiling

The header also declares `c_dbcsr_timeset` and `c_dbcsr_timestop`, which are DBCSR's Fortran-side timer callbacks. When profiling is enabled (`LIBXSTREAM_PROFILE_DBCSR`), every ACC function is bracketed by these calls so that individual backend operations appear in DBCSR's timing report.

Utility Macros

Macro	Description
`DBCSR_STRINGIFY(SYMBOL)`	Stringifies a preprocessor token
`DBCSR_CONCATENATE(A, B)`	Concatenates two preprocessor tokens
`DBCSR_MARK_USED(x)`	Silences unused-variable warnings

See Also

- LIBXSTREAM API (`libxstream/libxstream.h`) — the underlying OpenCL backend API
- DBCSR ACC specification — upstream interface definition

Visit LIBXS

OpenCL Backend

`libxstream/libxstream_opencl.h` is the internal OpenCL layer that powers every public `libxstream_*` function. It owns the OpenCL platform/device/context lifecycle, memory management, kernel compilation, and error handling. Sample code and other LIBXSTREAM extensions (e.g., LIBSMM, Ozaki) include this header to access the OpenCL runtime directly.

Macro	Default	Description

Compile-Time Configuration

The header is guarded by `__OPENCL` (set automatically when `__OFFLOAD_OPENCL` is defined). Key compile-time knobs:

Macro	Default	Description
`LIBXSTREAM_MAXALIGN`	2 MB	Maximum alignment for device allocations
`LIBXSTREAM_BUFFERSIZE`	8 KB	Internal scratch-buffer size
`LIBXSTREAM_MAXSTRLEN`	48	Maximum string length for names
`LIBXSTREAM_MAXNDEVS`	64	Maximum number of OpenCL devices
`LIBXSTREAM_MAXNITEMS`	1024	Per-thread maximum item count
`LIBXSTREAM_MAXNKERNELS`	32	Maximum number of distinct kernels that can be profiled
`LIBXSTREAM_PROFILE_TICKS`	10	Device-timer ticks a sample must span to be recorded
`LIBXSTREAM_USM`	SVM coarse-grain	Runtime Unified Shared Memory level (unset = OpenCL 2.0 SVM coarse-
`LIBXSTREAM_SUBBUFFER`	0	Sub-buffers for offset kernel-arguments (0 = off, 1 = on); unused where U

Levels 1 and 3 are opt-in and never reached by the default, even though both are faster in a microbenchmark. Level 1 (`cl_intel_unified_shared_memory`) is the only path with a genuinely asynchronous transfer — `clEnqueueMemcpyINTEL` measured 45.3 GB/s against 9.1 GB/s for a 128 MB H2D on a GPU Max 1550, where every SVM path instead ends in a host `memcpy` that runs single-threaded inside the enqueue and cannot overlap a kernel. It stays opt-in regardless, because the default must behave predictably across drivers rather than peak on one; request it explicitly where it is known good. A warning is emitted at verbosity 2+ if an explicit level 1 cannot load the extensions, since the only symptom is a slower run.

Level 3 is excluded from the default for a different reason. It adopts whatever `CL_DEVICE_SVM_CAPABILITIES` reports, and that varies by kernel driver: on i915 a GPU Max 1550 offers only coarse-grain buffers, whereas Xe also reports fine-grain, including fine-grain *system* allocations. That is a substantially broader contract than coarse-grain buffers and not something to acquire implicitly from a capability bit, so levels below 3 mask the capabilities down to `CL_DEVICE_SVM_COARSE_GRAIN_BUFFER`.

Nothing is lost by that: the grain is not a performance lever. Coarse-grain adds a `clEnqueueSVMMap`/`clEnqueueSVMUnmap` pair that fine-grain omits, but that is bookkeeping rather than data movement and the `memcpy` both paths end in dominates. Measured on a Xeon Platinum 8480+ (the device here reporting both grains), 64 MB H2D: 15.2 GB/s forced coarse against 15.1 GB/s with fine-grain.

On NVIDIA the capability is not queried by default, so the default level leaves USM off and no device memory pool is created. An explicitly requested level is honoured there, to exercise the path rather than to speed one up: results are correct and an H100 PCIe runs 6.7x slower that way. Leave it unset on that vendor.

Data Types

`libxstream_opencl_config_t` The central singleton (`libxstream_opencl_config`) populated by `libxstream_init`. It holds:

- **Device table** — ordered array of discovered `cl_device_id` entries.
- **Active device** (`libxstream_opencl_device_t`) — context, default stream, error slot, OpenCL standard level, workgroup limits, memory caps, vendor flags, and optional USM function pointers.
- **Resource pools** — lock objects, streams, events, memory-pointer registrations, and a host-memory pool (`libxs_malloc_pool_t`).

- **Runtime switches** — verbosity, async mode, debug/dump level, profiling, execution hints, and workaround level.
- **Histograms** — optional per-kernel duration histograms, and transfer-time histograms for H2D, D2H, D2D, and zero-fill operations.

`libxstream_opencl_stream_t` / `libxstream_event_t` Thin wrappers around `cl_command_queue` and `cl_event` respectively. Streams additionally carry a thread-ID and optional priority.

`libxstream_opencl_info_memptr_t` Associates a `cl_mem` buffer object with its host-side pointer, used to translate between SVM/USM pointers and buffer-based memory.

`libxstream_opencl_atomic_fp_t` Enumerates floating-point atomics support: none, 32-bit, or 64-bit.

Error Handling Macros

Macro	Description
`CL_CHECK(RESULT, CALL)`	Execute an OpenCL call; on failure record the error code and human-readable name
`CL_ERROR_REPORT(NAME)`	Print the last error to stderr (if verbosity is enabled)
`CL_RETURN(RESULT, NAME)`	Return from function, reporting the error if non-zero

Key Functions

Device and Context

Function	Description
`libxstream_opencl_set_active_device`	Internal device activation (lock-aware)
`libxstream_opencl_create_context`	Create an OpenCL context for a given device
`libxstream_opencl_device_name`	Return device name, platform name, and UID
`libxstream_opencl_device_level`	Query OpenCL version and device type
`libxstream_opencl_device_vendor`	Confirm a device's vendor string
`libxstream_opencl_device_ext`	Check for required OpenCL extensions
`libxstream_opencl_device_uid`	Capture or compute a unique device identifier
`libxstream_opencl_info_devmem`	Query free/total/local device memory

Memory

Function	Description
`libxstream_opencl_info_devptr`	Look up a device-pointer registration (read-only)
`libxstream_opencl_info_devptr_modify`	Look up a device-pointer registration (writable)
`libxstream_opencl_info_hostptr`	Look up a host-pointer registration
`libxstream_opencl_memset`	Fill device memory with an arbitrary byte pattern
`libxstream_opencl_use_cmem_size`	Whether OpenCL constant-memory hints apply
`libxstream_opencl_set_kernel_ptr`	Set a pointer kernel argument (USM-aware)

CUDA Host Registration Host memory from `libxstream_mem_host_allocate` is pinned by OpenCL, which a CUDA context cannot see: a transfer issued by CUDA from such a buffer is pageable and several times slower. Where the application links the CUDA runtime, `cudaHostRegister`/`cudaHostUnregister` are resolved at startup (global scope, i.e. no CUDA header and no link dependency) and such memory is registered when it is created and unregistered before it is given back. Registration is portable, so it also covers a context created later by `cudaSetDevice`.

Registration follows whichever path supplied the pages: a mapped buffer is registered individually, whereas USM/SVM memory is registered per pool block rather than per `libxs_malloc`, because the pool hands out pointers into a block and registration is page-granular. Blocks that came from plain `malloc` are excluded — those can share a page with unrelated heap data, and that configuration has no OpenCL device to begin with.

There is nothing to enable: a process without CUDA resolves nothing and pays nothing, and a process that links CUDA is one that intends to use it. `LIBXSTREAM_PIN=0` leaves the memory unregistered, which is how the pageable transport is measured. `LIBXSTREAM_PIN=2` goes further and loads the CUDA runtime when the application did not link it. That is off by default on purpose – loading a vendor runtime into a process that never asked for it initializes CUDA, can be slow or fail against a mismatched driver, and may collide with an application managing CUDA itself – but it is the setting a drop-in BLAS replacement needs, since such a process links `libOpenCL` and no CUDA runtime and would otherwise register nothing at all. `LIBXSTREAM_VERBOSE=2` reports how many allocations were offered and how many the CUDA runtime accepted; the partial case is the interesting one, since memory it refused stays pageable and reads as a slow kernel rather than as a slow copy.

Kernel Build

Function	Description
`libxstream_opencl_program`	Compile an OpenCL program from source, file, or binary
`libxstream_opencl_kernel_query`	Extract a named kernel from a compiled program
`libxstream_opencl_kernel`	Convenience: build + extract + release in one call
`libxstream_opencl_kernel_flags`	Assemble combined build flags from params, options, and extras
`libxstream_opencl_defines`	Merge user defines with internal definitions
`libxstream_opencl_flags_atomics`	Generate compiler flags for FP-atomic extensions

Streams, Events, and Timing

Function	Description
`libxstream_opencl_stream`	Find an existing stream for a thread-ID
`libxstream_opencl_stream_default`	Return the device's default (internal) stream
`libxstream_opencl_device_synchronize`	Per-thread device synchronization
`libxstream_opencl_launch`	Launch a kernel (drop-in for `clEnqueueNDRangeKernel`, profiling-aware)
`libxstream_opencl_duration`	Measure elapsed seconds from a `cl_event`

Error Utilities

Function	Description
`libxstream_opencl_strerror`	Map a `cl_int` error code to a string
`libxstream_opencl_error_consume`	Clear and return the last recorded error

Profiling

Two independent facilities, reported at exit on `stderr` and silent unless requested. They are separate because they measure different quantities in different units: a kernel has no byte count and hence no transfer rate, so mixing the rows would invite reading one as the other. Both may be set at once, which is the only case in which both appear.

Variable	Reports
`LIBXSTREAM_PROFILE`	Per-kernel durations (microseconds), one row per distinct kernel

Variable	Reports
`LIBXSTREAM_PROFILE_MEM`	Transfer rates (GB/s) for H2D, D2H, D2D, and zero-fill

A positive value sets the histogram resolution; a negative value additionally traces every individual sample as it is recorded. Setting either variable enables `CL_QUEUE_PROFILING_ENABLE` on all streams, so profiling is not meant for production runs.

Kernels are identified by `CL_KERNEL_FUNCTION_NAME`, read once per distinct `cl_kernel` handle. Call sites therefore pass no identifier: replacing `clEnqueueNDRangeKernel` with `libxstream_opencl_launch` is sufficient to make a kernel appear in the report.

Samples whose duration spans fewer than `LIBXSTREAM_PROFILE_TICKS` ticks of the device timer (`CL_DEVICE_PROFILING_TIMER_RESOLUT` are counted as discarded rather than recorded, because a rate derived from one or two ticks is quantization noise. The report states the timer resolution and the resulting floor only when samples were actually dropped, and reports `no samples recorded` when a requested profile collected nothing at all — silence there would be indistinguishable from a run that performed no work.

Every value a histogram carries is averaged per sample, never accumulated, so that a bucket's amount and its duration refer to the same single transfer and their ratio is a rate. Accumulating the duration instead divides a per-sample amount by a bucket total, which understates the rate by roughly the number of samples that share the bucket — and equally sized transfers all share one, so the error is largest exactly where the report is most useful.

The headline rate on a transfer row is the histogram's **mode**, not its median. Transfer sizes are commonly multi-modal — a whole operand alongside per-panel blocks, or a zero-fill covering both a small exponent array and a large slice plane — and a median can fall between the clusters and thus describe no observed transfer, whereas the mode always names a bucket that samples landed in. Where a workload issues transfers of one size the two agree, so nothing is lost where the median was already right. Kernel rows report the median duration.

Overlap A row gains one field, `inflight>=`, when its samples overlapped in time:

```
PROF ACC/OpenCL: ID=164984 ZERO=743.5 GB/s inflight>=1.08
PROF ACC/OpenCL: ID=164984 device inflight>=1.04 (3 kernels, 3 transfer kinds)
```

It is the average number of these operations running at once. Intervals cannot sum to more than the time they cover unless some of them ran together, so a ratio above 1 is overlap and nothing else.

The denominator is the **union** of the intervals — the time actually covered — not their extent, which would count the gaps between them. Each histogram folds its own intervals into that union as it goes (`libxs_hist_fold_union`), keeping only the segments a later interval could still merge with. An interval arriving after those have been retired is added whole, since its overlap with retired time can no longer be subtracted; such intervals are counted and the count is reported, and because that can only overstate the union, `inflight` stays a lower bound either way.

The field is absent whenever no overlap is demonstrated, which is the answer in itself: the operations ran one at a time. It is also absent when the overlap is too small to show in the two decimals reported, rather than printing a self-contradictory `1.00`.

A `device` row folds every kernel and every transfer into one union and states how many of each contributed. It reports what no single row can — one kernel overlapping another, or a transfer overlapping a kernel — because a union across them cannot be assembled from theirs: two rows may cover the same instant, and their totals cannot say whether they do.

A kernel whose every sample was rejected by the accuracy floor still gets a row, saying so, since it was launched and silence would say otherwise.

A negative value additionally traces every interval as `ns=<begin>-<end>` in device-clock nanoseconds, which is enough to reproduce any of this offline.

See Also

- LIBXSTREAM API (`libxstream/libxstream.h`) — public API built on top of this layer
- DBCSR ACC Interface — the DBCSR compatibility shim

Appendix

Presentations

- Stencils as Small Matrix Multiplications
- Practical Ozaki Schemes

Scripts

Helper scripts for building, inspecting, and maintaining LIBXSTREAM.

tool_checkabi.sh

Checks the ABI (Application Binary Interface) stability of the LIBXSTREAM shared library. The script uses `nm` to extract exported symbols from `lib/*.so` (or `lib/*.a` as fallback) and verifies that no previously published symbol has been removed or renamed compared to an earlier baseline (`.abi.txt`). Non-conforming symbol names cause an immediate error.

```
scripts/tool_checkabi.sh
```

NOTE: For full coverage the library must be built with `make STATIC=0 SYM=1` (or `DBG=1`) so that symbol information is present.

tool_getenvars.sh

Scans the LIBXSTREAM source tree (`src/*.c`) for calls to `getenv` and prints a sorted, deduplicated list of every environment variable used at runtime, separated into LIBXSTREAM-specific (`LIBXSTREAM_*`) and other variables.

```
scripts/tool_getenvars.sh
```

tool_opencl.sh

Converts OpenCL kernel files (`.cl`) — and optionally CSV parameter files — into a C header whose string literals can be compiled straight into the host binary. Include guards are stripped by default, and `#include` directives inside kernels are recursively resolved (inline).

```
scripts/tool_opencl.sh [options] infile.cl [infile2.cl ..] [infile.csv ..] outfile.h
```

Flag	Description
`-k`, `--keep`	Keep include guards (normally stripped)
`-b N`, `--banner N`	Copy the first N lines of the first `.cl` file as a banner
`-p DIR`, `--params DIR`	Directory containing CSV parameter files
`-c`, `-d`, `--debug`, `--comments`	Preserve comments in the generated source
`-v`, `--verbose`	Echo the command line

tool_version.sh

Derives the project version from Git tags and revision count. It can emit a plain version string, individual version components, or a complete C header with `#define` guards.

```
## full version string (branch-tag-patch)
scripts/tool_version.sh

## generate a C version header with a given symbol prefix
scripts/tool_version.sh LIBXSTREAM -1

## single component: 1=major, 2=minor, 3=update, 4+=patch
scripts/tool_version.sh LIBXSTREAM 2
```

Argument	Description
`PREFIX`	Symbol prefix used in the generated header (uppercased)
`CMPNT`	`0` (default) — version string; negative — C header; `1–3` — major/minor/update; `>3` — patch count
`SHIFT`	Added to the patch count (default: `0`)